

Number: P4167R0

Date: 2026-05-12

Audience: [Evolution](#) and [Library Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

-
- [Introduction](#)
 - [Disclaimer](#)
 - [Motivation](#)
 - [Fixing the example](#)
 - [Performance concerns](#)
 - [Combined function](#)
 - [Implementation](#)
 - [Shape of additional associated entities](#)
 - [Opt-out of ADL](#)
 - [ADL as a library code](#)
 - [Design options](#)
 - [Extending set of associated entities](#)
 - [Replacing associated entities set](#)
 - [Addition / Removal / Disable of all associated entities](#)
 - [Wording](#)
 - [Language wording](#)
 - [Library wording](#)
 - [Feature test macro](#)

Thanks to Lewis Baker for nerdsniping me.

P4167R0: reflection annotations to control ADL

Introduction

This paper provides a customization for ADL via an annotation to add an entity to be associated with a type. It adds a new `<meta>` type to represent the annotation, and it modifies [\[basic.lookup.argdep\]](#).

This paper is an exploration of a different design than proposed by similar paper [P2822R2](#), major difference is this paper doesn't propose new syntax and it uses existing annotation syntax.

Disclaimer

This paper is intended to start a discussion, doesn't have final wording, nor is consistent.

Motivation

Mateusz Pusz [presented](#) on an evening session in Croydon his [MP-Units library](#) paper, where he mentioned types with CNTTP are not associating their values as associated entities for purposes of ADL. Forcing users to explicitly qualify calls:

```
using namespace mp_units;
using namespace mp_units::si::unit_symbols;

namespace custom {
    // this is how MP-units defines a unit (so I define my own)
    inline constexpr struct awesomeness final : named_unit<"Awe", square(pi)> {} awesomeness;

    // and now I have a function which takes awesomeness per second...
    void process(quantity<awesomeness/s>);
}

void foo() {
    process((11 * custom::awesomeness) / (6 * s)); // before: fails to find right `process` function
                                                    // AFTER: will find `process` function in the `custom` namespace
}
```

You can experiment with the example on [compiler explorer](#). Major problem here is that quantity template is using value representation `awesomeness/s` which is not adding associated entities to ADL based overload resolution. This paper *doesn't propose adding adding CNTTP* (class nontype template parameters) *to ADL*, that would be breaking change. This paper proposes adding ability to add other associated entity via a custom annotation.

Fixing the example

So how we would fix the example? Type of `awesomeness` already has the custom namespace associated with itself. But problem is the quantity template and more precisely `derived_unit` template which is result of `awesomeness / sec` expression. We can modify the quantity first:

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity [[=-associated_entities(get_associated_entities_of(^R))]] {
    // ...
}
```

For a moment let's pretend we have `std::meta::get_associated_entities_of` and `std::meta::associated_entities` functions, they are found itself by ADL. First function `get_associated_entities_of` provides all associated namespaces / classes for purposes of ADL of type of `R`, and `associated_entities` provides annotation to quantity template instance.

Same modification will be needed in `derived_unit`, `scaled_unit`, ... and everywhere else where such transitive propagation is required for good user experience.

Performance concerns

This constant evaluation and reflection code is evaluated only during template instantiation, so it will be negligible. The code is not evaluated when an object instance of the type is created.

Combined function

I expect this pattern to be typical, so maybe it would be reasonable to provide a function which will do the introspection of provided reflection and build the annotation value directly.

```
template<Reference auto R, RepresentationOf<get_quantity_spec(R)> Rep = double>
class quantity [[=expand_associated_entities_from(^R)]] {
    // ...
}
```

Note: all names in this paper are placeholders.

Implementation

Currently none, only prototype on godbolt to gather the associated entities with purely library code. I will need to add the annotation inspection in ADL code.

Shape of `additional_associated_entities`

Special library type `additional_associated_entities` is designed to be stored in annotation. When compiler starts ADL, it can look at the annotation and if the object is there it can access to get additional entities. One possible approach could be:

```
struct additional_associated_entities {
    span<const meta::info> _data;
    consteval auto begin() const noexcept {
        return _data.begin();
    }
    consteval auto end() const noexcept {
        return _data.end();
    }
};
```

Compiler will then iterate the range of `meta::info` and adds each reflected entity into the associated entity list.

Opt-out of ADL

With similar approach we can provide opt-out of specific or all associated entities by providing two more types:

```
struct disassociated_entities {
    span<const meta::info> _data;
    consteval auto begin() const noexcept {
        return _data.begin();
    }
    consteval auto end() const noexcept {
        return _data.end();
    }
};

struct disabled_all_associated_entities {
    // just nothing
};
```

If annotations are remember in same order as declared, associating with only specific entities can be done via specifying these annotations in specific order. Alternatively more complex annotation type can be provided.

```
struct custom_associated_entities {
    bool disassociate_everything;
    span<const meta::info> disassociated;
    span<const meta::info> associated;
};
```

Or alternatively provide override mechanism:

```
struct associated_entities_override {
    span<const meta::info> _data;
    consteval auto begin() const noexcept {
        return _data.begin();
    }
    consteval auto end() const noexcept {
        return _data.end();
    }
};
```

ADL as a library code

Of course providing span of `meta::info` is cumbersome, so a helper functions are needed:

```

constexpr auto gather_associated_entities_of(meta::info entity) -> std::set<std::me
ta::info> {
    if (!is_value(entity)) entity = type_of(entity);
    if (!is_type(entity) && !is_namespace(entity)) {
        throw meta::exception("entity is not value, type, nor namespace", entity);
    }

    if (!has_parent(entity)) {
        return {};
    }

    // I know we don't have ordering or hashing here, but let's pretend we have
    std::set<meta::info> result{parent_of(entity)};

    // all parent's entities are associated
    result.insert_range(gather_associated_entities_of(parent_of(entity)));

    // process bases
    if (is_class_type(entity)) {
        for (meta::info bases: bases_of(entity)) {
            result.insert_range(gather_associated_entities_of(bases));
        }
    }

    if (has_template_arguments(entity)) {
        for (meta::info ta: template_arguments_of(entity)) {
            if (is_type(ta)) { // normal ADL ignores values, so we will too
                // TODO: make sure this is not infinite loop
                result.insert_range(gather_associated_entities_of(ta));
            }
        }
    }

    // process annotations
    // I process all types of override (just to show how straight-forward it is)
    for (meta::info annttns: annotations_of(entity)) {
        if (typeof(annttns) == ^^disabled_all_associated_entities) {
            result = {};
        } elseif (typeof(annttns) == ^^disassociated_entities) {
            for (meta::into negative: extract<disassociated_entities>(annttns)) {
                result.erase(negative);
            }
        } elseif (typeof(annttns) == ^^additional_associated_entities) {
            result.insert_range(extract<additional_associated_entities>(annttns));
        } else if (typeof(annttns) == ^^associated_entities_override) {
            // replace here
            result = extract<additional_associated_entities>(annttns) | std::ranges::t
o<std::set>
        }
    }
}

```

```

    return result;
}

constexpr auto override_associated_entities(std::set<meta::info> entity) {
    // obvious straightforward thing
    return associated_entities_override{define_static_array(entity)};
}

```

Design options

I'm putting design here after implementation, as it's more flexible. Basically we have multiple options what to provide:

- strict addition of associated entities,
- override of associated entities (with library responsible to gather otherwise lost associated entities on its own),
- addition, removal of some, and full removal of all already found (this requires keep the order of annotation or inventing a DSL).

Following subsection describes them in more details.

Extending set of associated entities

This is currently not worded, as I worded replacement. Will need addition of an annotation type `additional_associated_entities` which will contain a span of reflection for these entities. There is no need to provide a function to gether them, only a convinient constructor:

```

namespace std::meta {
    struct additional_associated_entities {
        // ...
        constexpr additional_associated_entities(std::initializer_list<std::meta::info>) { ... }
    }
}

// usage
struct my_type [=additional_associated_entities(^^custom)] { ... }

```

Replacing associated entities set

This is what I worded, it's the most powerful option. But it puts a responsibility on library to be in sync with language algorithm to gather associated entities.

Addition / Removal / Disable of all associated entities

Library will need to provide multiple annotations or a DSL to express this. It will make the language wording in [basic.lookup.argdep] much more complex.

Wording

We need to modify language part of ADL and then provide a few library types and functions, wording for the library part is not provided yet, based on what design is needed. Language change follows (probably wrong):

Language wording

6.5.4 Argument-dependent name lookup [basic.lookup.argdep]

- 1 When the *postfix-expression* in a function call ([`expr.call`]) is an *unqualified-id*, and unqualified lookup ([`basic.lookup.unqual`]) for the name in the *unqualified-id* does not find any
 - (1.1) — declaration of a class member, or
 - (1.2) — function declaration inhabiting a block scope, or
 - (1.3) — declaration not of a function or function template

then lookup for the name also includes the result of *argument-dependent lookup* in a set of associated namespaces that depends on the types of the arguments (and for type template template arguments, the namespace of the template argument), as specified below.

[*Example 1:*

```
namespace N {
    struct S { };
    void f(S);
}

void g() {
    N::S s;
    f(s);           // OK, calls N::f
    (f)(s);         // error: N::f not considered; parentheses prevent argument-d
                    // ependent lookup
}
```

— *end example*]

- 2 [Note 1: For purposes of determining (during parsing) whether an expression is a *postfix-expression* for a function call, the usual name lookup rules apply. In some cases a name followed by `<` is treated as a *template-name* even though name lookup did not find a *template-name* (see [temp.names]). For example,

```
int h;
void g();
```

```
namespace N {
    struct A {};
    template <class T> int f(T);
    template <class T> int g(T);
    template <class T> int h(T);
}
```

```
int x = f<N::A>(N::A());           // OK, lookup of f finds nothing, f treated as te
template name
int y = g<N::A>(N::A());           // OK, lookup of g finds a function, g treated as t
template name
int z = h<N::A>(N::A());           // error: h< does not begin a template-id
```

The rules have no effect on the syntactic interpretation of an expression. For example,

```
typedef int f;
namespace N {
    struct A {
        friend void f(A &);
        operator int();
        void g(A a) {
            int i = f(a);           // f is the typedef, not the friend function: equi
valent to int(a)
        }
    };
}
```

Because the expression is not a function call, argument-dependent name lookup does not apply and the friend function `f` is not found. — *end note*

- 3 For each argument type `T` in the function call, there is a set of zero or more *associated entities* to be considered. The set of entities is determined entirely by the types of the function arguments (and any type template arguments). Any *typedef-names* and *using-declarations* used to specify the types do not contribute to this set. The set of entities is determined in the following way:

- (3.1) — If `T` is `std::meta::info` ([[meta.syn](#)]), its associated set of entities is the singleton containing the enumeration type `std::meta::operators` ([[meta-reflection.operators](#)]).

[*Note 2:* The `std::meta::info` type is a type alias, so an explicit rule is needed to associate calls whose arguments are reflections with the namespace `std::meta`. — *end note*]

- (3.2) — If `T` is any other fundamental type, its associated set of entities is empty.
- (3.?) — If `T` has an annotation of type `associated_entities_override`, associated entities are provided only by iterating the annotation object to get their reflection in form of objects of type `std::meta`.

[*Note ?:* This is intended as an override mechanism for ADL.. — *end note*]

- (3.3) — If *T* is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and, if it is a complete type, its direct and indirect base classes. Furthermore, if *T* is a class template specialization, its associated entities also include: the entities associated with the types of the template arguments provided for template type parameters; the templates used as type template template arguments; and the classes of which any member templates used as type template template arguments are members.

[*Note 3*: Constant template arguments, variable template template arguments, and concept template arguments do not contribute to the set of associated entities. — *end note*]

- (3.4) — If *T* is an enumeration type, its associated entities are *T* and, if it is a class member, the member's class.
- (3.5) — If *T* is a pointer to *U* or an array of *U*, its associated entities are those associated with *U*.
- (3.6) — If *T* is a function type, its associated entities are those associated with the function parameter types and those associated with the return type.
- (3.7) — If *T* is a pointer to a member function of a class *X*, its associated entities are those associated with the function parameter types and return type, together with those associated with *X*.
- (3.8) — If *T* is a pointer to a data member of class *X*, its associated entities are those associated with the member type together with those associated with *X*.

In addition, if the argument is an overload set or the address of such a set, its associated entities are the union of those associated with each of the members of the set, i.e., the entities associated with its parameter types and return type. Additionally, if the aforementioned overload set is named with a *template-id*, its associated entities also include its template template arguments and those associated with its type template arguments.

- 4 The *associated namespaces* for a call are the innermost enclosing non-inline namespaces for its associated entities as well as every element of the inline namespace set ([[namespace.def](#)]) of those namespaces. Argument-dependent lookup finds all declarations of functions and function templates that

- (4.1) — are found by a search of any associated namespace, or
- (4.2) — are declared as a friend ([[class.friend](#)]) of any class with a reachable definition in the set of associated entities, or
- (4.3) — are exported, are attached to a named module *M* ([[module.interface](#)]), do not appear in the translation unit containing the point of the lookup, and have the same innermost enclosing non-inline namespace scope as a declaration of an associated entity attached to *M* ([[basic.link](#)]).

If the lookup is for a dependent name ([`temp.dep`], [`temp.dep.candidate`]), the above lookup is also performed from each point in the instantiation context ([`module.context`]) of the lookup, additionally ignoring any declaration that appears in another translation unit, is attached to the global module, and is either discarded ([`module.-global.frag`]) or has internal linkage.

5 [Example 2:

Translation unit #1:

```
export module M;
namespace R {
    export struct X {};
    export void f(X);
}
namespace S {
    export void f(R::X, R::X);
}
```

Translation unit #2:

```
export module N;
import M;
export R::X make();
namespace R { static int g(X); }
export template<typename T, typename U> void apply(T t, U u) {
    f(t, u);
    g(t);
}
```

Translation unit #3:

```
module Q;
import N;
namespace S {
    struct Z { template<typename T> operator T(); };
}
void test() {
    auto x = make();           // OK, decltype(x) is R::X in module M
    R::f(x);                   // error: R and R::f are not visible here
    f(x);                      // OK, calls R::f from interface of M
    f(x, S::Z());              // error: S::f in module M not considered
                                // even though S is an associated namespace
    apply(x, S::Z());          // error: S::f is visible in instantiation context,
                                // but
                                // R::g has internal linkage and cannot be use
                                // d outside TU #2
}
```

— end example]

6 [Note 4: The set of associated namespaces can include namespaces already considered by ordinary unqualified lookup. — end note]

[*Example 3:*

```
namespace NS {  
    class T { };  
    void f(T);  
    void g(T, int);  
}  
NS::T parm;  
void g(NS::T, float);  
int main() {  
    f(parm);                                // OK, calls NS::f  
    extern void g(NS::T, float);  
    g(parm, 1);                             // OK, calls g(NS::T, float)  
}
```

— *end example*]

Library wording

Will provide wording for function `override_associated_entities`, `gather_associated_entities_of`, and other functions.

Feature test macro

15.11 Predefined macro names [cpp.predefined]

`__cpp_adl_override` 202???